
CHAPINWARRIORSSA: SISTEMA AUTÓNOMO PARA LA PLANIFICACIÓN DE MISIONES DE RESCATE Y EXTRACCIÓN EN CIUDADES

202504863 - Kevin Ánderson Montejo Martínez

Resumen

ChapinWarriorsSA es una aplicación de escritorio desarrollada en C# con .NET que permite simular misiones autónomas de rescate y extracción de recursos en ciudades representadas mediante mallas de celdas. La información de las ciudades y robots se obtiene desde archivos XML, permitiendo configurar diferentes escenarios sin modificar directamente el código de la aplicación. Para encontrar las rutas de las misiones, el sistema transforma la malla de la ciudad en un grafo ortogonal y utiliza el algoritmo de búsqueda en anchura (BFS) desde los diferentes puntos de entrada. Las reglas de movimiento dependen del tipo de robot: ChapinRescue debe evitar las unidades militares para rescatar civiles, mientras ChapinFighter puede enfrentarlas cuando posee una capacidad de combate mayor, reduciendo su capacidad después de cada combate. Además, el sistema utiliza Graphviz para representar gráficamente las rutas y generar reportes de las misiones. La implementación permitió aplicar programación orientada a objetos, estructuras de datos y manejo de archivos, obteniendo una solución modular que facilita agregar nuevas ciudades, robots y misiones.

Abstract

ChapinWarriorsSA is a desktop application developed in C# with .NET 10 that simulates autonomous rescue and resource extraction missions in cities represented by cell grids. City and robot information is obtained from XML files, allowing different scenarios to be configured without directly modifying the application code. To find mission routes, the system transforms the city grid into an orthogonal graph and uses the Breadth-First Search (BFS) algorithm from the different entry points. Movement rules depend on the type of robot: ChapinRescue must avoid military units to rescue civilians, while ChapinFighter can confront them when it has a higher combat capacity, reducing its capacity after each combat. In addition, the system uses Graphviz to graphically represent the routes and generate mission reports. The implementation made it possible to apply object-oriented programming, data structures, and file management, resulting in a modular solution that facilitates adding new cities, robots, and missions.

Palabras clave.

XML, BFS, Grafos, Graphviz, Robots autónomos

Keywords: XML, BFS, Graphs, Graphviz, Autonomous robots

Introducción

El desarrollo de sistemas capaces de encontrar soluciones de forma automática es una parte importante de la programación, especialmente cuando se trabaja con problemas que requieren tomar decisiones sobre diferentes caminos. El proyecto ChapinWarriorsSA plantea este problema mediante misiones de rescate y extracción de recursos en ciudades representadas como mallas de celdas. En estas ciudades existen caminos, entradas, obstáculos, civiles, recursos y unidades militares que afectan las rutas que pueden seguir los robots.

El propósito de este ensayo es explicar cómo se construyó una solución para este problema utilizando C#, programación orientada a objetos, estructuras de datos, archivos XML, grafos y el algoritmo BFS. También se analizará cómo las reglas de cada tipo de robot influyen en la búsqueda de una ruta válida y cómo la aplicación presenta los resultados mediante Graphviz. De esta forma, se busca mostrar cómo diferentes conceptos vistos durante el curso pueden integrarse para resolver un problema de programación completo.

Imagen 1. Portada del proyecto

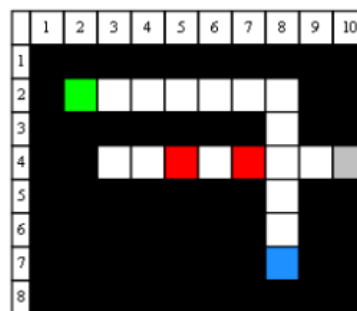


Fuente: Elaboración propia

Desarrollo del tema.

1. Modelado de ciudades mediante matrices y grafos. La ciudad es uno de los elementos principales de ChapinWarriorsSA y se representa mediante una malla organizada en filas y columnas. Cada celda puede representar un camino, un punto de entrada, una unidad civil, un recurso, una unidad militar o una posición intransitable. Esta representación permite que el sistema conozca qué posiciones puede recorrer un robot y cuáles debe evitar.

Imagen 2. Ejemplo de ciudad



Fuente: Elaboración propia

A partir de esta malla, el sistema construye un grafo ortogonal donde cada celda se relaciona con sus posiciones vecinas: arriba, abajo, izquierda y derecha. De esta forma, la ciudad puede ser utilizada como un conjunto de nodos y conexiones para realizar la búsqueda de rutas.

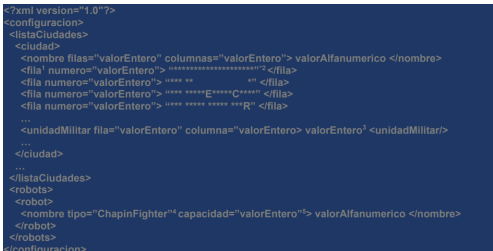
Esta transformación permite separar la información de la ciudad de la lógica utilizada para resolver las misiones. Así, cuando se carga una nueva ciudad desde un archivo XML, el sistema puede construir su grafo automáticamente y utilizarlo para encontrar las rutas sin modificar el algoritmo principal.

2. Lectura de archivos XML y administración

de datos. ChapinWarriorsSA utiliza archivos XML para almacenar la información de las ciudades y los robots. Estos archivos contienen datos como el nombre y tamaño de cada ciudad, el contenido de sus filas, las posiciones de las unidades militares y las características de los robots. Esto permite configurar diferentes escenarios sin tener que modificar directamente el código de la aplicación.

En la aplicación, el usuario puede seleccionar la opción “Cargar XML” para que el sistema revise los archivos de configuración disponibles y cargue su información. Cuando existen ciudades o robots con nombres repetidos, los datos se actualizan en lugar de generar duplicados, cumpliendo con uno de los requerimientos establecidos para el sistema.

Imagen 3. Ejemplo archivo XML



Fuente: Instructivo del proyecto [IPC2]Proyecto_1_202602.pdf

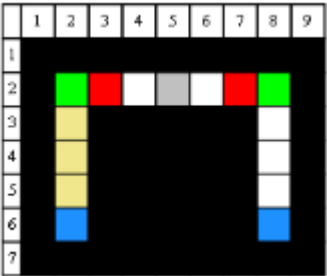
El uso de XML facilita la separación entre los datos y la programación. Esto permite agregar nuevos mapas o robots sin cambiar la lógica principal de las misiones, haciendo que el sistema sea más flexible y fácil de ampliar.

3. Algoritmo BFS multi-fuente para la

planificación de rutas. Para encontrar las rutas de las misiones se utiliza el algoritmo Breadth-First Search (BFS), o búsqueda en anchura. En ChapinWarriorsSA se inicia la búsqueda utilizando los diferentes puntos de entrada de la ciudad, permitiendo encontrar una ruta válida desde cualquiera de ellos hacia el destino seleccionado.

Las reglas de la búsqueda dependen del robot. ChapinRescue debe evitar las unidades militares y buscar un camino seguro hasta una unidad civil. En cambio, ChapinFighter puede atravesar unidades militares cuando su capacidad de combate es mayor que la de estas, pero al derrotarlas pierde una cantidad de capacidad igual a la capacidad de combate de la unidad derrotada. Por lo tanto, BFS no solamente busca una conexión entre dos puntos, sino que debe respetar las condiciones de cada tipo de misión. Si encuentra una ruta que cumple las reglas, esta se utiliza para ejecutar la misión y mostrar el resultado. Si no existe una ruta válida, el sistema indica que la misión es imposible.

Imagen 4. Ejemplo camino encontrado



Fuente: Elaboración propia

4. Arquitectura orientada a objetos y organización del sistema. ChapinWarriorsSA fue desarrollado utilizando programación orientada a objetos, separando las responsabilidades principales de la aplicación. La lógica del sistema se encuentra organizada en componentes como XMLReader, MissionEngine y OrthogonalGrid, mientras que la navegación y las ventanas de la aplicación se manejan mediante NavigationController y las diferentes vistas. Esta separación permite que cada parte del programa tenga una función específica.

La aplicación también utiliza estructuras de datos desarrolladas para almacenar y manejar la información cargada desde los archivos XML. Esto permite trabajar con las ciudades, robots y demás elementos del proyecto sin depender de las estructuras propias de C# que estaban restringidas en el proyecto.

Esta organización facilita el mantenimiento del sistema, ya que modificar una parte específica no implica cambiar toda la aplicación. Por ejemplo, agregar nuevas ciudades mediante XML no requiere modificar el algoritmo encargado de realizar las misiones. De esta manera, la solución puede crecer sin afectar su funcionamiento principal.

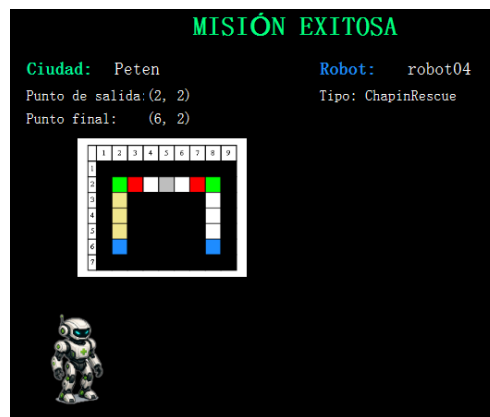
5. Visualización de resultados y generación de reportes. Además de encontrar las rutas, el sistema debe mostrar al usuario los resultados de las misiones de una manera clara. Para esto se utiliza Graphviz, herramienta que permite generar una representación gráfica de la ciudad y resaltar

el camino encontrado por el robot. El uso de esta herramienta forma parte de los requerimientos del proyecto.

Al finalizar una misión, la aplicación muestra información como el resultado de la misión, el punto de salida utilizado, el destino y el estado del robot. En el caso de un ChapinFighter, también se puede observar cómo cambió su capacidad de combate después de enfrentarse a unidades militares.

Los resultados también se almacenan en un historial de misiones que puede ser consultado posteriormente. De esta forma, el sistema no solamente ejecuta la misión, sino que permite revisar lo ocurrido y visualizar nuevamente la ruta generada. Esto facilita comprender el resultado y comprobar el funcionamiento del algoritmo.

Imagen 5. Ejemplo de Reporte



Fuente: Elaboración propia

6. Aprendizajes y aportes del proyecto.

El desarrollo de ChapinWarriorsSA permitió aplicar diferentes conceptos vistos durante el curso en una misma solución. Entre ellos se encuentran la programación orientada a objetos, estructuras de datos, manejo de archivos XML,

grafos y algoritmos de búsqueda. La combinación de estos elementos permitió pasar de un problema planteado de forma general a una aplicación capaz de ejecutar misiones de manera automática.

Uno de los principales aprendizajes fue comprender cómo un problema puede dividirse en partes más pequeñas para facilitar su solución. La separación entre la lectura de datos, construcción del mapa, búsqueda de rutas y presentación de resultados permitió trabajar cada componente de manera independiente y después integrarlos en una sola aplicación.

Finalmente, el proyecto también permitió aplicar el uso de GitHub para llevar un control del desarrollo mediante diferentes versiones o *releases*. Esto ayudó a mantener un registro de los cambios realizados durante el proyecto y forma parte de los requerimientos establecidos para comprobar el avance de la solución.

Conclusiones.

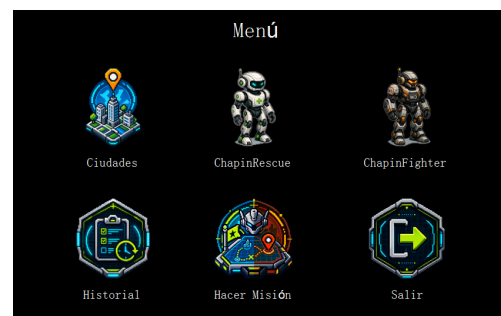
El desarrollo de ChapinWarriorsSA permitió comprobar cómo diferentes conceptos de programación pueden combinarse para resolver un problema completo. La utilización de grafos y BFS permitió crear un sistema capaz de buscar rutas automáticamente tomando en cuenta las condiciones de cada misión y el tipo de robot utilizado.

El uso de archivos XML permitió separar la información de las ciudades y robots de la lógica de la aplicación. Esto hizo posible trabajar con

diferentes escenarios sin modificar el algoritmo principal. De igual manera, la programación orientada a objetos y la separación de responsabilidades facilitaron la organización del código y permitieron que las diferentes partes del sistema trabajen de forma independiente.

Finalmente, el proyecto permitió comprender que una solución de software no consiste únicamente en hacer que el programa funcione, sino también en organizar correctamente sus componentes y presentar los resultados de forma comprensible. La integración de estructuras de datos, algoritmos, XML, Graphviz y control de versiones permitió construir una aplicación que cumple con las funciones planteadas y que puede ser ampliada con nuevos escenarios y misiones.

Imagen 6. Menú Interactivo



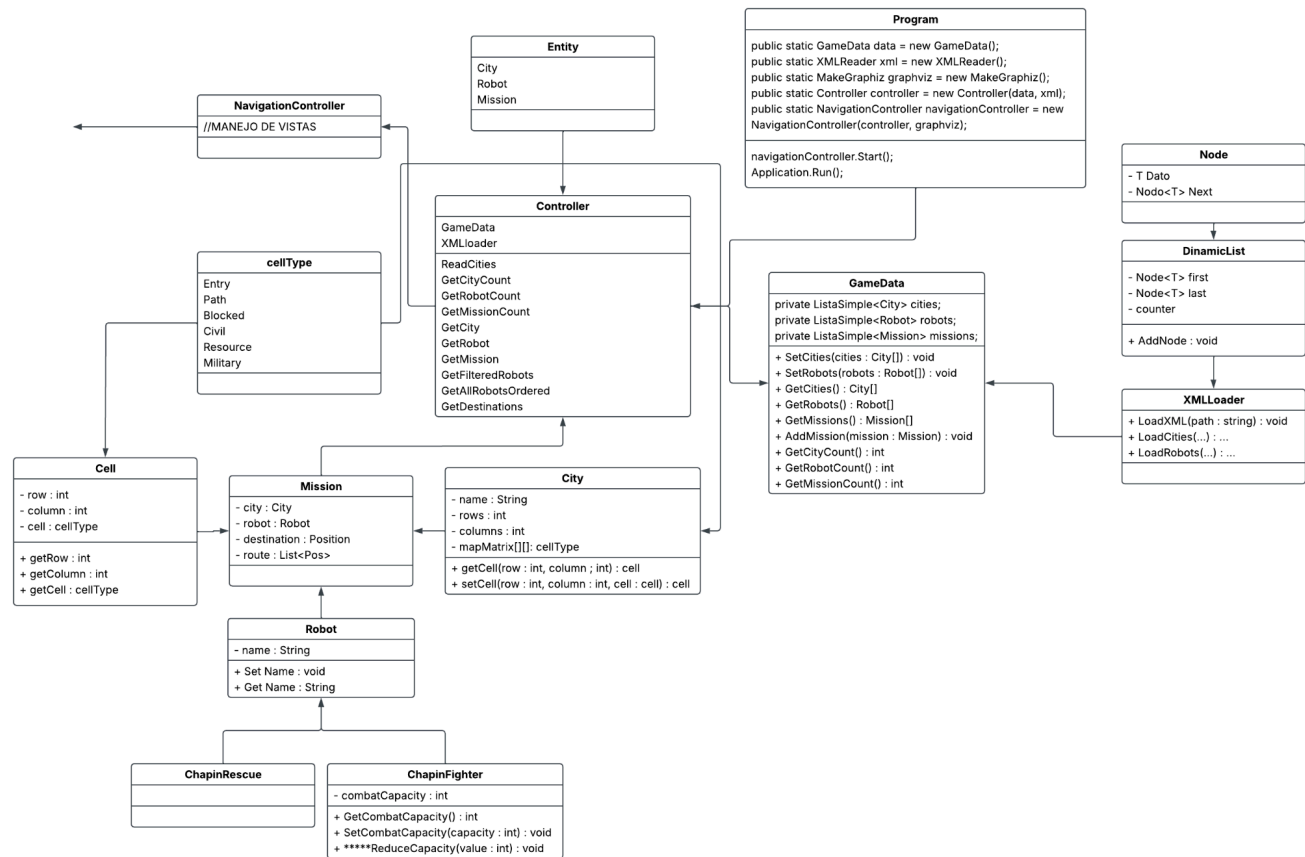
Fuente: Elaboración propia

Bibliografía:

- *What is Orthogonal Matrix? Examples, Properties, Determinant.* (2026). Cuemath. https://www-cuemath-com.translate.goog/algebra/orthogonal-matrix/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true
- Tech, M. (2026). *Mentores Tech.* Mentores Tech. <https://www.mentorestech.com/resource-algorithms-breadth-first-search.php>

Diagrama de Clases.

Imagen 6. Menú Interactivo



Fuente: Elaboración propia

https://lucid.app/lucidchart/68482c23-d533-4194-9a8e-84bbb0ee8bd1/edit?viewport_loc=-2100%2C-1434%2C3850%2C1949%2C0_0&invitationId=inv_4a8f74d4-8a9a-4874-ad36-c8e2e99230e3